

```

test_name_function.py:5: TypeError
===== short test summary info =====
FAILED test_name_function.py::test_first_last_name - TypeError:
    get_formatted_name() missing 1 required positional argument: 'last'
===== 1 failed in 0.04s =====

```

There's a lot of information here because there's a lot you might need to know when a test fails. The first item of note in the output is a single F ❶, which tells us that one test failed. We then see a section that focuses on FAILURES ❷, because failed tests are usually the most important thing to focus on in a test run. Next, we see that `test_first_last_name()` was the test function that failed ❸. An angle bracket ❹ indicates the line of code that caused the test to fail. The E on the next line ❺ shows the actual error that caused the failure: a `TypeError` due to a missing required positional argument, `last`. The most important information is repeated in a shorter summary at the end, so when you're running many tests, you can get a quick sense of which tests failed and why.

Responding to a Failed Test

What do you do when a test fails? Assuming you're checking the right conditions, a passing test means the function is behaving correctly and a failing test means there's an error in the new code you wrote. So when a test fails, don't change the test. If you do, your tests might pass, but any code that calls your function like the test does will suddenly stop working. Instead, fix the code that's causing the test to fail. Examine the changes you just made to the function, and figure out how those changes broke the desired behavior.

In this case, `get_formatted_name()` used to require only two parameters: a first name and a last name. Now it requires a first name, middle name, and last name. The addition of that mandatory middle name parameter broke the original behavior of `get_formatted_name()`. The best option here is to make the middle name optional. Once we do, our test for names like Janis Joplin should pass again, and we should be able to accept middle names as well. Let's modify `get_formatted_name()` so middle names are optional and then run the test case again. If it passes, we'll move on to making sure the function handles middle names properly.

To make middle names optional, we move the parameter `middle` to the end of the parameter list in the function definition and give it an empty default value. We also add an `if` test that builds the full name properly, depending on whether a middle name is provided:

```

name
_function.py def get_formatted_name(first, last, middle=''):
    """Generate a neatly formatted full name."""
    if middle:
        full_name = f"{first} {middle} {last}"
    else:
        full_name = f"{first} {last}"
    return full_name.title()

```
